SYMMETRY IN DEEP NEURAL NETWORKS

COBY MULLIKEN

Abstract. A classic question in the study of deep learning is whether distinct neural nets trained on similar datasets learn similar representations of those data. Several recent papers have probed this question using tools from group theory and representation theory using model stitching—that is, joining the head of one model to the tail of another—and examining the performance of the resultant model. This gives a sense of the 'compatibility' of two models' learned representations. Another key idea is the notion of equivariance, which provides a framework for producing relative representations of model latent spaces that are more 'universal' than their absolute counterparts. Here we review the key results of this research program and present our own model stitching experiments with natural data.

1. Introduction

A classic question in machine learning research is whether deep neural networks of similar architecture trained on similar data learn similar representations of that data in their latent spaces. [7] [8] More precisely, if we take φ to be the first ℓ layers of a k-layer network f, which we will call the *representer*, and ψ' to be the last $k-\ell$ layers of a different network f', then under what conditions can we find a relatively simple transformation s such that the network $\psi' \circ s \circ \varphi$ performs about as well as f or f'?

One answer to this question is the 'Anna Karenina hypothesis' of [1], which posits—taking from the first sentence of Tolstoy's *Anna Karenina*—that all 'good' representations are similar, but that each 'bad' representation is 'bad' in its own way. How ought we test this hypothesis? A natural answer comes from the framework of model stitching, in which we split two networks at a given layer ℓ and attach them with some simple transformation s, so as to produce the most coherent output possible given the constraints. If a simple layer is capable of stitching a representer and learner with minimal loss (compared to a homogeneous network), then we can conclude that the two networks have, in some sense, 'compatible' representations.

We will see, in this paper, how far we can stretch this line of thinking. We will see that narrowing down our possible space of stitching layers using tools from algebra can provide us with more a more detailed picture of 'how convergent' these latent space representations are. Finally, we will motivate the notion of a *relative representation*, which promises to provide a consistent description of latent space representations across models.

Date: December 11, 2024.

2. Background

2.1. **Model Stitching.** Let us begin by clarifying our notation for neural networks.[1] We follow, for the most part, the conventions of [5]. We denote a neural network $f : \mathbb{R}^{n_0} \to \mathbb{R}^{n_k}$ as $f(\cdot, W, \sigma)$, where $\sigma : \mathbb{R} \to \mathbb{R}$ is an activation function applied to vectors coordinate-wise, and W is a vector consisting of the weights and biases of f, so that

$$W = (W_1, b_1, W_2, b_2, \ldots, W_k, b_k).$$

When σ is clear from the context, we will simply write $f(\cdot, W)$. We denote by $f_i : \mathbb{R}^{n_i} \to \mathbb{R}^{n_{i+1}}$ the action of the ith layer, so that f is the composition of all of its layers:

$$f_i(x) = \sigma(W_i x + b_i) \qquad f(x) = f_k \circ f_{k-1} \circ \cdots \circ f_0(x).$$

We can then describe a stitching layer at layer ℓ as $s : \mathbb{R}^{n_\ell} \to \mathbb{R}^{n_{\ell+1}}$. Our goal, then, in the model stitching framework, is to choose a 'simple' class of functions $\mathcal{S}$ over which to optimize, so as to minimize some loss function $\mathcal{L} : \mathcal{F} \to \mathbb{R}$, where $\mathcal{F}$ is the space of functions parametrized by neural nets of our architecture.

More precisely, if we are stitching the representer of f to the classifier of g by splitting at layer ℓ, we can write

$$\mathcal{L}_\ell(f; g) = \inf_{s \in \mathcal{S}} \mathcal{L}(g_{>\ell} \circ s \circ f_{\leq \ell})$$

Where the left-hand quantity is defined as the *stitching penalty* of f and g at layer ℓ. The reason we want $\mathcal{S}$ to be 'simple,' in some appropriately defined way, is so that it is only powerful enough to adapt the features learned by $f_{\leq \ell}$ to the features 'expected' by $g_{>\ell}$, rather than learning new features of its own. In essence, we would like s to 'plug in' the features learned by $f_{\leq \ell}$ into $g_{>\ell}$, and do no more. Why do we need the stitching layer in the first place? The following provides a useful motivating example.

Example 1. *Let f be a neural network of depth k, and let P be a permutation matrix. Let f' be identical to f, except that the weights and biases are permuted consistently at each layer by some set of permutation matrices $\{P_i\}_{i=1}^k$. Then f and f' are equal as functions, and at layer i have identical representations up to permutation, but the model $f'_{>i} \circ f_{\leq i}$ is likely incoherent.*

The class of stitching layers proposed by [1] is that of affine transformations. (In the case of the MLP, this is involves the addition of a single, linear layer between $f_{\leq \ell}$ and $h_{>\ell}$.) We can easily optimize over this class via gradient descent. We will see in **Section 4** that a single affine layer can effectively stitch two different models, but fails to learn well given a random representation, lending credence to the 'Anna Karenina hypothesis.'

2.2. **Weight Space Symmetries.** Thus far, the questions we have formulated have been primarily empirical in nature. A reasonable goal would be to try and formalize these questions in group theoretic language—that is, to reframe our study as that of the invariances of neural networks as functions under transformations to their weights. This is precisely what [2] [5] [4] [7] endeavor to do. Let us first introduce some basic terminology.

[1] For simplicity, we will only discuss multilayer perceptrons (MLPs). All of the theory we will develop, however, also applies to convolution neural networks (CNNs) and, under some restrictions, residual neural networks as well (RNNs).

Definition 2.3. *Let $x \in \mathcal{X} \subset \mathbb{R}^d$, and let $\varphi : \mathcal{X} \to \mathcal{Z}$ be a representation. We say φ is* ***equivariant*** *with respect to a function $g : \mathcal{X} \to \mathcal{Z}$ if there exists a map M_g such that*

$$\varphi(g(x)) = M_g(\varphi(x)) \qquad \forall x \in \mathcal{X}$$

We say φ is ***invariant*** *to the action of g if M_g is the identity map. Finally, we say two representations φ and φ' are* ***equivalent*** *if there exists a map $E_{\varphi\to\varphi'} : \mathcal{Z} \to \mathcal{Z}$ such that*

$$E_{\varphi\to\varphi'}(\varphi(x)) = \varphi'(x) \qquad \forall x \in \mathcal{X}.$$

The key insight of [5] is to introduce a group of transformations on the weight space called the *intertwiner group*.

Definition 2.4. *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a function that can be extended to $\mathbb{R}^n$ by applying σ coordinate-wise, and let $GL_n(\mathbb{R})$ be the group of invertible linear operators on $\mathbb{R}^n$. Then we define*

$$G^n_\sigma = \{A \in GL_n(\mathbb{R}) \mid \exists B \in GL_n(\mathbb{R}) : \sigma \circ A = B \circ \sigma\}$$

And we call G^n_σ the ***intertwiner group*** *corresponding to σ.*

Intuitively, we can think of G^n_σ as the set of invertible linear transformations on $\mathbb{R}^n$ whose action composed with that of σ can be recovered by applying another invertible linear transformation *after* applying σ.

Theorem 2.5. *Suppose $\sigma(I_n)$ is invertible. Then G^n_σ is a group. Furthermore, if we define $\phi_\sigma(A) = \sigma(A)\sigma(I_n)^{-1}$ for $A \in GL_n(\mathbb{R})$, then ϕ_σ is a homomorphism from G^n_σ to $GL_n(\mathbb{R})$ and $\sigma \circ A = \phi_\sigma(A) \circ \sigma$.* [5]

Proof. We have immediately that $G^n_\sigma \subset GL_n(\mathbb{R})$, so it suffices to show that for any $A, B \in G^n_\sigma$, $AB^{-1} \in G^n_\sigma$. We also have $\sigma \circ A = A' \circ \sigma$ and $\sigma \circ B = B' \circ \sigma$ for some $A', B' \in GL_n(\mathbb{R})$. We then have

$$\sigma \circ AB^{-1} = A' \circ \sigma \circ B^{-1} \qquad \sigma = B' \circ \sigma \circ B^{-1}$$

From which we deduce that $(B')^{-1} \circ \sigma = \sigma \circ B^{-1}$. Combining yields

$$\sigma \circ AB^{-1} = A'(B')^{-1} \circ \sigma$$

Making G^n_σ a subgroup of $GL_n(\mathbb{R})$. □

We note that for $\sigma = \text{ReLU}$, the case of most interest to us, G^n_σ consists of all matrices of the form PD, where P is a permutation matrix and D is a diagonal matrix with positive entries. We will denote this group by G_{ReLU}. This group provides a natural means for describing the symmetries, or invariances, of ReLU activated MLPs. In specific, if we consider the *realization map* $\Phi : \mathcal{W} \to \mathcal{F}$, where $\mathcal{W}$ is the space of possible weights for neural networks of our architecture, which satisfies $\Psi : W \mapsto f(\cdot, W)$, we have the following equivalence due to [5].

Proposition 1. *Let $W \in \mathcal{W}$ be a weight vector $(W_1, b_1, \ldots, W_k, b_k)$ and let $A_i \in G^{n_i}_\sigma$ for $1 \le i < k$, so that*

$$W' = (A_1W_1, A_1b_1, A_2W_2\phi_\sigma(A_1^{-1}), A_2b_2, \ldots, W_k\phi_\sigma(A_{k-1}^{-1}), b_k)$$

Then $\Phi(W') = \Phi(W)$ as functions.

The promise of the intertwiner group is that it provides an even 'simpler'—indeed, better motivated—class of stitching layers over which to optimize. The ReLU-activated case provides particularly good intuition: since G^n_{ReLU} consists of matrices PD, we can think of transformations in G^n_{ReLU} as scaling and permuting weights, without mixing them. This seems, on the surface, to be a rather naive method of 'stitching'; and yet, we have the following result, again due to [5].

Theorem 2.6. *Let W' be an appropriately transformed copy of W as before. If we let $\mathcal{R}$ be an empirical risk functional with respect to some loss function, and decompose $f(\cdot, W) = \gamma \circ \varphi$ and $f(\cdot, W') = \gamma' \circ \varphi'$ at layer ℓ, then*

$$\mathcal{R}[f(\cdot, W)] = \mathcal{R}[f(\cdot, W')] = \mathcal{R}[\gamma \circ s \circ \varphi']$$

Is achievable for $s \in \mathcal{S}$ if $\mathcal{S}$ contains $\phi_\sigma(G^n_\sigma)$.

Using this guarantee—under the rather large assumption that two networks have similar enough weights—to stitch two networks using G_{ReLU} we may simply optimize over all matrices PD using the convex relation between permutation matrices and doubly stochastic matrices.

2.7. **Relative Representations.** We have seen thus far various methods for stitching models and analyzing their weight space symmetries. Another good goal would be to develop a method for representing similar latent spaces in an appropriately 'coordinate-free' fashion, such that similar representations produce similar outputs under this method. Let us formalize this.

Given an input space $\mathcal{X}$, let us choose a collection of points $\mathcal{A} \subset \mathcal{X}$ which we will call the **anchor** set. Ideally, we would like this set to contain samples from all the classes in our data (or if our data is unlabeled, then we would like it to be sufficiently 'diverse' in some appropriately defined way). [9] We then have the following representation.

Definition 2.8. *Given $\mathcal{A} = (a_1, \ldots, a_r) \subset \mathcal{X}$ and a similarity function* sim $: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$*, a* ***relative representation*** *of $x \in \mathcal{X}$ with respect to $\mathcal{A}$ is*

$$\mathcal{T}^{\mathcal{A}}_{rel}(x) = (\mathrm{sim}(x, a_1), \ldots, \mathrm{sim}(x, a_r)).$$

If we choose cosine similarity as our similarity function, such that $\mathrm{sim}(a, b) = \frac{a \cdot b}{\|a\|\|b\|}$ then we have the following equivariance due to [4].

Theorem 2.9. *Let* sim *be cosine similarity, and denote by $\mathcal{A}'$, z' the images of $\mathcal{A}, z$ under the transformation $\alpha : z \mapsto PDz + h$ for some permutation matrix P, diagonal matrix D, and fixed vector h. Then*

$$\mathcal{T}^{\mathcal{A}}_{rel}(z) = \mathcal{T}^{\mathcal{A}'}_{rel}(z').$$

We can then construct stitched models which utilize relative representations as their stitching layer. We will discuss the practical implications of the framework in **Section 5**.

3. Formulation

To investigate the convergence of latent space representations, we constructed several modes of stitching and relative representation and analyzed their accuracy numerically and heuristically.

Our first experiment aimed to test the stitching capacity of a basic multilayer perceptron (MLP) model trained for the task of pixel regression on a single image. The model consisted of five linear layers, with 300 neurons each, and ReLU activation. Stitching layers were accordingly also 300-neuron hidden layers, inserted after the activation function for the ℓth layer. For each layer, two different architectures were tested. The first consisted of training two neural networks f and f' of identical architecture but different initializations on the same training set (every other pixel) for ten epochs each, and then taking $f_{<\ell}$ and $f'_{\geq\ell}$, freezing their weights, and inserting the stitching layer s to produce a new network $f'_{\geq\ell} \circ s \circ f_{<\ell}$, which was then trained for ten epochs on the same trainset (updating only the weights of s).

The second consisted of plugging a trained $f'_{\geq\ell}$ into an untrained, randomly initialized $g_{<\ell}$, and then freezing the weights of both and training a stitching layer s to minimize the loss of $f'_{\geq\ell} \circ s \circ g_{<\ell}$. This was intended to ensure that the stitching layer was not 'learning' on its own, but rather adapting the features of f to those expected by f'. That is, we wanted to avoid a scenario in which a stitching layer produced reasonable results even given an incoherent representation by constructing *its own* good representation. We quantified our results using the notion of a stitching penalty, introduced in **Section 1**.

Our second investigation focused on the latent space of a simple autoencoder. Namely, in the vein of [9], we wanted to see if geometric or topological patterns emerged in an autoencoder's representation of its inputs in a low-dimensional latent space. The autoencoders were trained on the MNIST handwritten digit database, and consisted of four ReLU-activated 100-neuron hidden layers in the encoder, followed by a two-dimensional bottleneck with batch normalization, and an analogous decoder. We then implemented a simple relative representation with cosine similarity. We attempted two versions: the first used only ten anchors—one drawn from each class—whereas the second used 100. For the 100-anchor case, we used principal component analysis to visualize the results.

The aim of this experiment was to see if relative representations were capable of exposing the convergent nature of the features learned by differently intialized networks. If the relative representations learned were similar across initializations, then we would expect that to indicate that the 'absolute' representations learned were all *essentially* the same—that is, equivalent up to simply geometric transformations like scaling, rotation, and translation.

4. Results

In accordance with the results of [1], we found overwhelmingly positive evidence for the efficacy of stitching with a non-activated linear layer optimized by gradient descent. Figure 1 shows the results of pixel regression using several different model architectures explored; we note that while the stitching layer trained with a randomly initialized representer produces a blurry and noisy result, the stitching of two differently trained networks produces a reasonably sharp—if undetailed—rendering of the original image. (We opted not to use Random Fourier Features, which would have siginificantly increased the detail of the regressed image, in order to simplify the process and ensure we were only studying the most basic properties of feature extraction in a nautral setting.) This suggests that the stitching layer itself was incapable of learning a rich representation, relying on a 'good enough' representation to be provided by prior layers. However, the trained, stitched model did perform

better (visually-speaking) than the unstitched, trained model. A reasonable explanation for this is that the extra training time (and size) provided by the stitching layer might have marginally increased performance.

We can quantify this more precisely. In Figure 2, we note that the stitched models consistently attain *negative* stitching penalties. This should not be altogether surprising, considering that the gains are rather modest; we are, after all, adding a layer to our model and giving it more training time. Even if the layer is simple, it is understandable that it might further optimize our already optimized head and tail models. A more proper comparison would be the trained, stitched, heterogeneous model (that is, head and tail trained separately) with a trained, stitched, homogeneous model (that is, head and tail trained together). Still, we note that the model with an untrained representer performs consistently worse than the trained model, with a disparity that increases as the stitching is moved later in the model. (This finding is consistent with [1].)

These findings were complemented by our findings on the latent spaces of autoencoders. We note in Figure 3 that each of the four pictured latent spaces contain *roughly* similar structure, despite being initialized randomly and independently. For example, we notice similar patterns in terms of the adjacencies of certain classes. (The batch normalization ensures the centeredness of the latent space clusters.) However, the correspondences are not perfect; some adjacencies vary widely between initializations, further complicated by the fact that none of the models perfectly separate the classes.

To make our questions more precise, we can examine the representations produced by each model, and make them relative to a fixed set of anchors. In Figure 4, we see how this approach (using 10 anchors, one from each class) results in *somewhat* consistent results. If models converged perfectly in their representations, then we would expect the relative representations to be more or less constant for each input. This is not exactly what we observe. Still, we note that columns (within each plot) are fairly consistent in value, indicating that for many inputs, the relative representations are fairly consistent. This line of inquiry merits further investigation.

5. Conclusion

We have given here a cursory look at how symmetry provides a useful angle for probing the internal structure of deep neural networks: by splicing and stitching models and identifying invariances, we gain a more precise sense of the dynamics of their learning. Stitching is of more than theoretical interest: [9] recently used relative representation-based zero-shot model stitching to adapt a well-trained language model for other languages. If we can identify the unchanging parts of a model's latent space across initialization, then we can potentially adapt previously trained models for novel tasks with minimal extra training. [10]

Unfortunately, it has been rather outside our scope to describe the more fundamental reasons for the results we have described. However, we will attempt to give a brief overview of research directions on this topic here. One reason to think convergent representations are rather unsurprising is that [3] showed that neural network loss landscapes typically consist of wide, flat valleys, as opposed to narrow local minima or fractal-like wells. This suggests that stochastic gradient descent

FIGURE 1. Stitched network performance on pixel regression task. Trained representation consists of network $\gamma' \circ s \circ \varphi$, where both γ' and φ are trained. Random representation consists of $\gamma \circ s \circ \varphi_r$, where φ_r is randomly initialized and untrained. Unstitched is $\gamma \circ \varphi$ where γ and φ are trained together.

might naturally select models separated by a low-loss path through one of these 'loss valleys.' [2]

Further work might better quantify—in a probabilistic manner—the frequency and magnitude of convergence of model representations; that is, how often—and under what conditions—do models produce similar latent representations? Moreover, can we motivate or deduce these convergences from more basic principles, as [6] suggest? And finally, how can we exploit these weight space symmetries to improve model performance? The possible research agenda is long but, as I hope we've hinted at, quite fruitful.

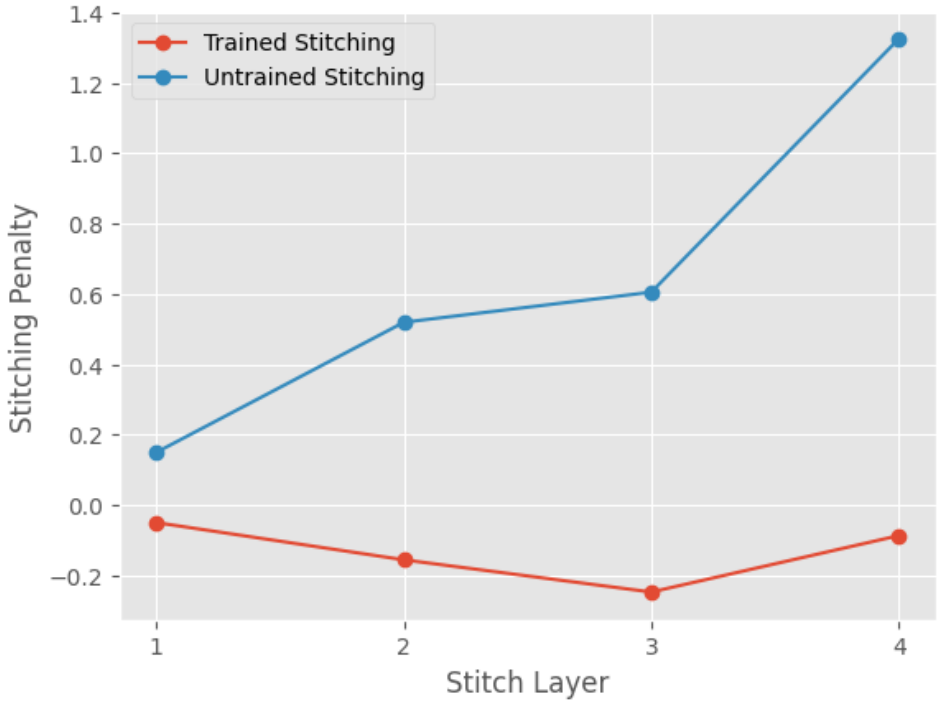

FIGURE 2. Stitching penalties for stitching at various layers. Trained stitching is a two trained components stitched and then trained, whereas untrained stitching is a randomly initialized representer and a trained head stitched and then trained.

REFERENCES

[1] Yamini Bansal, Preetum Nakkiran, and Boaz Barak, *Revisiting model stitching to compare neural representations*, 2021.

[2] Johanni Brea, Berfin Simsek, Bernd Illing, and Wulfram Gerstner, *Weight-space symmetry in deep networks gives rise to permutation saddles, connected by equal-loss valleys across the loss landscape*, 2019.

[3] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht, *Essentially no barriers in neural network energy landscape*, 2019.

[4] Alejandro García-Castellanos, Giovanni Luca Marchetti, Danica Kragic, and Martina Scolamiero, *Relative representations: Topological and geometric perspectives*, 2024.

[5] Charles Godfrey, Davis Brown, Tegan Emerson, and Henry Kvinge, *On the symmetries of deep learning models and their internal representations*, 2023.

[6] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola, *The platonic representation hypothesis*, 2024.

[7] Karel Lenc and Andrea Vedaldi, *Understanding image representations by measuring their equivariance and equivalence*, 2015.

[8] Yixuan Li, Jason Yosinski, Jeff Clune, Hod Lipson, and John Hopcroft, *Convergent learning: Do different neural networks learn the same representations?*, 2016.

[9] Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà, *Relative representations enable zero-shot latent space communication*, 2023.

[10] Haotian Ye, Yihong Liu, Chunlan Ma, and Hinrich Schütze, *Mosecrot: Model stitching with static word embeddings for crosslingual zero-shot transfer*, 2024.

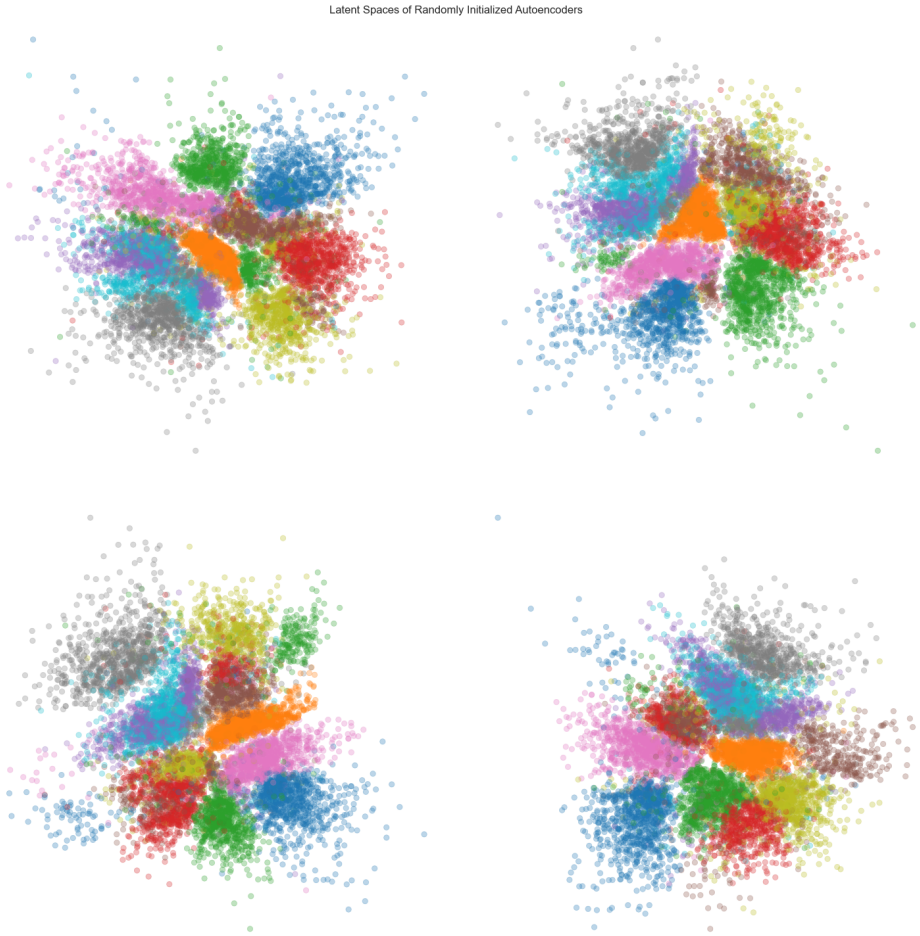

FIGURE 3. Representation of MNIST test set in 2-dimensional latent spaces of several randomly initialized autoencoders.

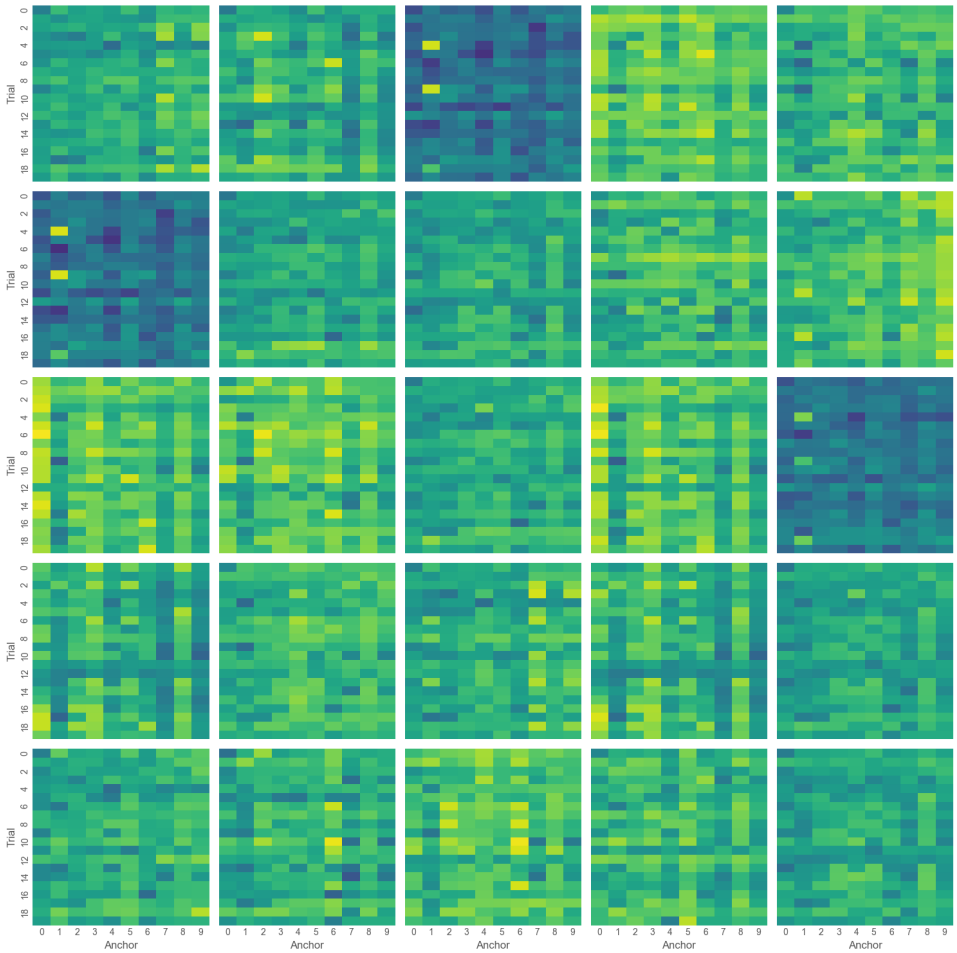

FIGURE 4. Relative representations of 25 different sample images over 20 trials. Relative representations are with respect to ten anchors, one from each class, given on x-axis.